
Database Design & SQLAlchemy (Theory)

Welcome to MySQL! In this session, we transition from stateless APIs to persistent data storage. You will learn the fundamentals of relational databases, how to set up MySQL and MySQL Workbench, core SQL commands (DDL and DML), and how to bridge Python code with databases using **SQLAlchemy 2.0 ORM**.

1. Relational Database Fundamentals & MySQL Setup

A. What is a Relational Database?

A relational database stores data in structured tables consisting of rows (records) and columns (attributes).

- **Tables:** Represent entities (e.g., `users`, `predictions`).
- **Primary Key:** A unique identifier for every row in a table.
- **Foreign Key:** A column that links to the primary key of another table, establishing relationships.

B. MySQL & MySQL Workbench Installation

MySQL is one of the world's most popular open-source relational database management systems. **MySQL Workbench** is a visual tool used for database design, SQL development, and administration.

1. Go to the official [MySQL Downloads Page](#).
 2. Download and run the **MySQL Installer** for your operating system.
 3. Choose the **Developer Default** or **Server + Workbench** setup type.
 4. Set a secure root password during configuration and keep note of it for your connection string later.
-

2. SQL Fundamentals: DDL and DML

SQL (Structured Query Language) is used to interact with relational databases. It is divided into two primary categories relevant to our backend apps:

A. DDL (Data Definition Language)

DDL commands define and modify the database schema structure:

- `CREATE DATABASE` : Sets up a new database schema.
- `CREATE TABLE` : Defines new tables, columns, data types, and constraints.
- `ALTER TABLE` : Modifies an existing table structure.
- `DROP TABLE` : Deletes an entire table and its data.

Example (SQL):

```
CREATE DATABASE ml_backend_db;
```

```
USE ml_backend_db;
```

```
CREATE TABLE predictions (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    feature1 FLOAT NOT NULL,  
    feature2 FLOAT NOT NULL,  
    prediction_result FLOAT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

B. DML (Data Manipulation Language)

DML commands manage the data stored *inside* those tables:

- `INSERT INTO` : Adds new rows of data.
- `SELECT` : Queries and retrieves data.
- `UPDATE` : Modifies existing records.
- `DELETE` : Removes specific rows.

Example (SQL):

```
-- Insert a record  
INSERT INTO predictions (feature1, feature2, prediction_result) VALUES  
(1500.0, 3.0, 2255.50);
```

```
-- Query records  
SELECT * FROM predictions;
```

3. Object-Relational Mapping (ORM) with SQLAlchemy 2.0

Writing raw SQL queries inside Python applications can become complex and prone to security issues (like SQL injection). **ORM (Object-Relational Mapping)** solves this by allowing you to interact with your database using pure Python classes and objects.

A. What is SQLAlchemy 2.0?

SQLAlchemy is the premier Python SQL toolkit and ORM. Version 2.0 introduced a modernized, Pythonic style featuring strict type hinting, native support for modern Python features, and cleaner session management.

- **Models as Classes:** In SQLAlchemy, a database table is represented as a Python class, and each column is an attribute of that class.
- **Declarative Base:** A base class that keeps track of all your database models.

Conceptual Example (SQLAlchemy 2.0):

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class PredictionModel(Base):
    __tablename__ = "predictions"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    feature1: Mapped[float] = mapped_column(nullable=False)
    feature2: Mapped[float] = mapped_column(nullable=False)
    prediction_result: Mapped[float] = mapped_column(nullable=False)
```

4. Reference Links & Further Reading

- **MySQL Documentation:** Comprehensive guides on MySQL server administration and SQL syntax.
 - **MySQL Workbench Guide:** Official documentation for visual database design and querying.
 - **SQLAlchemy 2.0 Official Tutorial:** The definitive guide for mastering modern Python ORM patterns.
-